

主成分分析

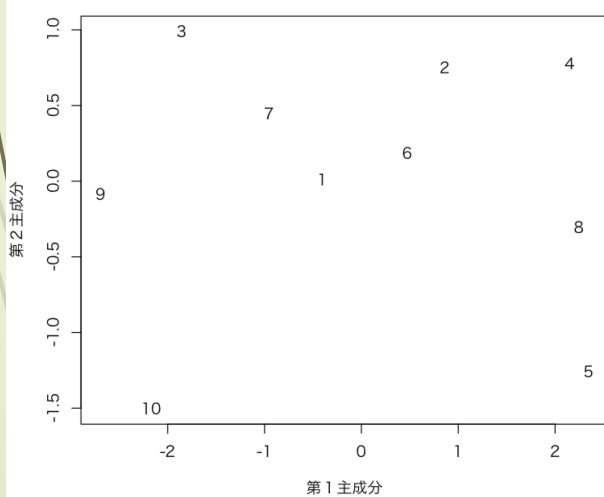
多変量データ (複数の変数からなるデータ)

教科試験データ

生徒 No.	国語 x_1	社会 x_2	数学 x_3	理科 x_4	英語 x_5
1	58	49	58	74	57
2	47	42	50	59	45
3	47	39	54	69	49
4	53	47	65	79	57
5	52	53	48	65	56
6	43	35	44	60	43
7	48	42	47	65	51
8	54	49	64	80	49
9	44	40	55	70	43
10	39	37	55	65	47

主成分分析とは

多変量データに含まれる主要な情報を縮約した新たな変数（主成分）を作成し、多変量データを視覚化する統計的方法



先の5教科試験データに対して主成分分析を行い, 5変数（国語, 社会, 理科, 社会, 英語）を情報を2変数（第1主成分, 第2主成分）の情報に縮約し, 10人の生徒の相対的関係を散布図上に視覚化

多変量データの表記

例：5教科試験データ

生徒 No.	国語 x_1	社会 x_2	数学 x_3	理科 x_4	英語 x_5
1	x_{11}	x_{12}	x_{13}	x_{14}	x_{15}
2	x_{21}	x_{22}	x_{23}	x_{24}	x_{25}
3	x_{31}	x_{32}	x_{33}	x_{34}	x_{35}
4	x_{41}	x_{42}	x_{43}	x_{44}	x_{45}
5	x_{51}	x_{52}	x_{53}	x_{54}	x_{55}
6	x_{61}	x_{62}	x_{63}	x_{64}	x_{65}
7	x_{71}	x_{72}	x_{73}	x_{74}	x_{75}
8	x_{81}	x_{82}	x_{83}	x_{84}	x_{85}
9	x_{91}	x_{92}	x_{93}	x_{94}	x_{95}
10	x_{101}	x_{102}	x_{103}	x_{104}	x_{105}

原データのスケーリング

2種類のスケーリング

1. 中心化（各変数の平均を引く）

$$x'_{ij} = x_{ij} - \bar{x}_j$$

2. 標準化（各変数の平均を引いて、標準偏差で割る）

$$x'_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}$$

x_{ij} : i 番目の個体の j 番目の変数の値

$\bar{x}_j$: j 番目の変数の平均

s_j : j 番目の変数の標準偏差

主成分の合成

ここでは、元の変数が x_1, x_2 の2変数であるとし、中心化によってスケーリングされているものとする。

この2変数に a_1, a_2 の重みをかけて新たな変数 f_1 を合成する

$$f_{i1} = a_1 x'_{i1} + a_2 x'_{i2}, \quad i = 1, \dots, n$$

重み a_1, a_2 は、 f_1 の分散

$$\text{Var}(f_1) = \frac{\sum_{i=1}^n (f_{i1} - \bar{f}_1)^2}{n-1}$$

が最大になるように決める。

分散 = 情報

x_{i1}, x_{i2} には実際のデータが入るので、分散は、 a_1, a_2 の関数であることに注意
最初に作成する主成分 f_1 は、最も主要な主成分なので **第1主成分** と呼ばれる

第2主成分の合成

次に2番目の主成分 f_2 を合成する

$$f_{i2} = b_1 x'_{i1} + b_2 x'_{i2}, \quad i = 1, \dots, n$$

第2主成分 f_2 の重み b_1, b_2 も f_1 と同様に, 分散

$$\text{Var}(f_2) = \frac{\sum_{i=1}^n (f_{i2} - \bar{f}_2)^2}{n-1}$$

が最大になるように決める.

ただし, 第2主成分 f_2 は, 第1主成分 f_1 と無相関であるという条件が加わる

$$\text{無相関: } \frac{\sum_{i=1}^n (f_{i1} - \bar{f}_1)(f_{i2} - \bar{f}_2)}{\sqrt{\sum_{i=1}^n (f_{i1} - \bar{f}_1)^2} \sqrt{\sum_{i=1}^n (f_{i2} - \bar{f}_2)^2}} = 0$$

One point : 固有値・固有ベクトル (1)

$p \times p$ 正方行列 C があるときに、

$$Cx = \lambda x$$

となるような, スカラー λ を固有値, ベクトル x を固有ベクトルと呼ぶ

固有値・固有ベクトルの例

$$C = \begin{pmatrix} 4 & \sqrt{3} \\ \sqrt{3} & 2 \end{pmatrix} \text{とすると}$$

$$\begin{pmatrix} 4 & \sqrt{3} \\ \sqrt{3} & 2 \end{pmatrix} \begin{pmatrix} -0.866 \\ -0.500 \end{pmatrix} = 5 \begin{pmatrix} -0.866 \\ -0.500 \end{pmatrix}$$

が成り立つので, 固有値, $\lambda = 5$, 固有ベクトル, $x = \begin{pmatrix} -0.866 \\ -0.500 \end{pmatrix}$

$$4 \times -0.866 + \sqrt{3} \times -0.500 = 5 \times -0.866$$

$$\sqrt{3} \times -0.866 + 2 \times -0.500 = 5 \times -0.500$$

One point : 固有値・固有ベクトル (2)

$p \times p$ 正方行列 C があるときに、

$$Cx = \lambda x$$

となるような、固有値 λ , 固有ベクトル x の組は最大で p 組とすることが出来る

先の行列 $C = \begin{pmatrix} 4 & \sqrt{3} \\ \sqrt{3} & 2 \end{pmatrix}$ に対して、

$\lambda = 1, x = \begin{pmatrix} 0.500 \\ -0.866 \end{pmatrix}$ も固有値, 固有ベクトルであり、

$$\begin{pmatrix} 4 & \sqrt{3} \\ \sqrt{3} & 2 \end{pmatrix} \begin{pmatrix} 0.500 \\ -0.866 \end{pmatrix} = 1 \begin{pmatrix} 0.500 \\ -0.866 \end{pmatrix}$$

が成り立つ

したがって、 $\lambda_1 = 5, x_1 = \begin{pmatrix} -0.866 \\ -0.500 \end{pmatrix}, \lambda_2 = 1, x_2 = \begin{pmatrix} 0.500 \\ -0.866 \end{pmatrix}$ となり、2組とれている。

主成分の重みの計算 (1)

第1主成分の重み a_1, a_2 は、共分散行列の最大固有値に対する固有ベクトル

第2主成分の重み b_1, b_2 は、2番目に大きな固有値に対する固有ベクトル

共分散行列： (i, j) 要素に i 番目の変数と j 番目の変数の間の共分散

$$Cov(x_i, x_j) = \frac{\sum_{k=1}^n (x_{ki} - \bar{x}_i)(x_{kj} - \bar{x}_j)}{n - 1}$$

が格納されている行列

※変数 x_i と変数 x_i の間の共分散 $Cov(x_i, x_i)$ は、分散となる

$p = 2$ の時の共分散行列

$$C = \begin{pmatrix} Cov(x_1, x_1) & Cov(x_1, x_2) \\ Cov(x_2, x_1) & Cov(x_2, x_2) \end{pmatrix}$$

共分散行列の計算例

～ 5教科試験データ ～

- subjectのデータは, 行がサンプル, 列が変数になるので, cov関数の引数rowvarをFalseにし, サンプル間ではなく, 変数間の分散共分散行列を計算する.
- subjectの列見出し (columns) を分散共分散行列の行見出し (index), と列見出し (columns) に設定する.

```
import pandas as pd
import numpy as np

subject = pd.read_csv("subject.csv", index_col=0)
cov = np.cov(subject, rowvar=False)
df_cov = pd.DataFrame(cov, index=subject.columns, columns=subject.columns)
print(df_cov)
```

	国語	社会	数学	理科	英語
国語	33.166667	29.500000	19.111111	26.555556	24.500000
社会	29.500000	34.900000	16.666667	22.688889	25.322222
数学	19.111111	16.666667	48.888889	46.555556	14.666667
理科	26.555556	22.688889	46.555556	52.711111	20.311111
英語	24.500000	25.322222	14.666667	20.311111	29.788889

共分散行列の固有値・固有ベクトルの計算例

～ 5教科試験データ ～

numpy のlinalgパッケージのeig関数で固有値・固有ベクトルを計算する.

```
e_val, e_vec = np.linalg.eig(df_cov)
print("固有値: {}".format(e_val))
print("固有ベクトル")
print(e_vec)
```

```
固有値: [141.38838586  42.82332732  7.57000962  2.76368848  4.91014428]
固有ベクトル
[[ 0.41011839  0.37713381  0.37873347 -0.46924901  0.57091096]
 [ 0.39262023  0.47970175  0.38371133  0.27668604 -0.62605594]
 [ 0.49283415 -0.54962164 -0.06701551 -0.54380162 -0.39347081]
 [ 0.56126787 -0.38702139  0.03094701  0.63811812  0.35642722]
 [ 0.34604211  0.41926938 -0.83897424 -0.01831016  0.01596871]]
```

左の例では, 固有値をe_val, 固有ベクトルをe_vecとしている. 最大固有値は, 141.39であり, 最大固有ベクトルに対する固有ベクトルは, e_vecの1列目になる.

主成分得点の計算 (1)

```
e_val,e_vec=np.linalg.eig(df_cov)
print("固有値:{}".format(e_val))
print("固有ベクトル")
print(e_vec)
ave_sub=subject.mean()
print(ave_sub)
```

固有値: [141.38838586 42.82332732 7.57000962 2.76368848 4.91014428]

固有ベクトル

```
[[ 0.41011839  0.37713381  0.37873347 -0.46924901  0.57091096]
 [ 0.39262023  0.47970175  0.38371133  0.27668604 -0.62605594]
 [ 0.49283415 -0.54962164 -0.06701551 -0.54380162 -0.39347081]
 [ 0.56126787 -0.38702139  0.03094701  0.63811812  0.35642722]
 [ 0.34604211  0.41926938 -0.83897424 -0.01831016  0.01596871]]
```

国語 48.5
社会 43.3
数学 54.0
理科 68.6
英語 49.7
dtype: float64

生徒 No.	国語 x_1	社会 x_2	数学 x_3	理科 x_4	英語 x_5
1	58	49	58	74	57

	国語 $\bar{x}_1$	社会 $\bar{x}_2$	数学 $\bar{x}_3$	理科 $\bar{x}_4$	英語 $\bar{x}_5$
平均	48.5	43.3	54.0	68.6	49.7

したがって No.1の生徒に対する第1主成分得点は

$$f_{11} = a_1x'_{11} + a_2x'_{12} + a_3x'_{13} + a_4x'_{14} + a_5x'_{15}$$

$$= 0.4101 \times (58 - 48.5) + 0.3926 \times (49 - 43.3) + 0.4928 \times (58 - 54.0) + 0.5613 \times (74 - 68.6) + 0.5613 \times (57 - 49.7)$$

$$= 13.6624$$

主成分得点の計算 (2)

5科目試験データの共分散行列の
固有ベクトル

固有ベクトル

```
[[ 0.41011839  0.37713381  0.37873347 -0.46924901  0.57091096]
 [ 0.39262023  0.47970175  0.38371133  0.27668604 -0.62605594]
 [ 0.49283415 -0.54962164 -0.06701551 -0.54380162 -0.39347081]
 [ 0.56126787 -0.38702139  0.03094701  0.63811812  0.35642722]
 [ 0.34604211  0.41926938 -0.83897424 -0.01831016  0.01596871]]
```

↑
行列 A
(因子負荷量)

個々の個体やサンプルに対する
主成分の値（主成分得点）は次の様に計算される

$$f_{i1} = a_1x'_{i1} + a_2x'_{i2}$$

$$f_{i2} = b_1x'_{i2} + b_2x'_{i2} \quad i = 1, \dots, n$$

これを行列で表すと

$$X = \begin{pmatrix} x'_{11} & x'_{12} \\ x'_{21} & x'_{22} \\ \vdots & \vdots \\ x'_{n1} & x'_{n2} \end{pmatrix}, A = \begin{pmatrix} a_1 & b_1 \\ a_2 & b_2 \end{pmatrix}, f = \begin{pmatrix} f_{11} & f_{12} \\ f_{21} & f_{22} \\ \vdots & \vdots \\ f_{n1} & f_{n2} \end{pmatrix}$$

主成分得点の計算は、行列を使って表すと

$$f = XA$$

主成分得点の計算 (3)

主成分得点の計算例 (5科目試験データの例)

```
from sklearn import preprocessing as pp
ss = pp.StandardScaler(with_std=False)
ctr_subject=ss.fit_transform(subject)
pr_sc=np.dot(ctr_subject,e_vec)
print(pr_sc)
```

```
[[ 13.66235049  5.08933563 -0.44033767 -1.74378794  2.32253058]
 [-10.11148998  2.7540128  2.84722479 -3.52048807 -1.96536475]
 [-2.32116689 -3.07671537 -1.61839814 -0.21781212  1.96706689]
 [ 17.08269649 -3.53819537 -3.41580118 -0.5669357 -0.25207188]
 [ 2.44632665 13.30547927  0.0527131  1.89171359 -2.89626454]
 [-17.58812635 -0.04026501  0.75730016  0.35725381  2.81869765]
 [-5.73601412  4.9734995 -1.42115865  1.36051555  2.02033422]
 [ 15.57815218 -5.39321285  4.54011141  0.84558837 -0.31112441]
 [-4.18105247 -7.18067438  2.62688975  1.67079841 -0.50457774]
 [-8.83167602 -6.89326423 -3.92854357 -0.07684589 -3.19922602]]
```

- sklearnのpreprocessingモジュールのStandardScalerクラスを使ってデータの中心化をしておく。StandardScalerは、デフォルトで標準化を行うが、中心化の場合は、with_std=Falseとする。
- Numpyのdot関数を使って、行列ctr_subjectと行列e_vecの行列せきを計算する。np.dot(a,b)で行列aと行列bの行列積を行う
- 主成分得点の計算結果の行が各サンプル（個体）、列が主成分を表す
- 例えば 1番目の個体の第1主成分得点は、13.66235049

主成分の解釈 (1)

固有ベクトル

```
[[ 0.41011839  0.37713381  0.37873347 -0.46924901  0.57091096]
 [ 0.39262023  0.47970175  0.38371133  0.27668604 -0.62605594]
 [ 0.49283415 -0.54962164 -0.06701551 -0.54380162 -0.39347081]
 [ 0.56126787 -0.38702139  0.03094701  0.63811812  0.35642722]
 [ 0.34604211  0.41926938 -0.83897424 -0.01831016  0.01596871]]
```

国語	社会	数学	理科	英語
x_1	x_2	x_3	x_4	x_5

$x'_i = x_i - \bar{x}_i$ $i = 1, \dots, 5$ なので
平均以下だと負の値になる

第1主成分の計算式は、

$$f_1 = a_1 x'_1 + a_2 x'_2 + a_3 x'_3 + a_4 x'_4 + a_5 x'_5 \\ = 0.413 \times x'_1 + 0.393 \times x'_2 + 0.493 \times x'_3 + 0.561 \times x'_4 + 0.346 \times x'_5$$

a_1, a_2, a_3, a_4, a_5 全ての係数が正の値

全科目が高得点 → 第1主成分 f_1 の値が大きい
全科目が低得点 → 第1主成分 f_1 の値が小さい

第1主成分
： 総合能力の尺度

主成分の解釈 (2)

固有ベクトル

```
[ [ 0.41011839  0.37713381  0.37873347 -0.46924901  0.57091096]
 [ 0.39262023  0.47970175  0.38371133  0.27668604 -0.62605594]
 [ 0.49283415 -0.54962164 -0.06701551 -0.54380162 -0.39347081]
 [ 0.56126787 -0.38702139  0.03094701  0.63811812  0.35642722]
 [ 0.34604211  0.41926938 -0.83897424 -0.01831016  0.01596871]]
```

国語	社会	数学	理科	英語
x_1	x_2	x_3	x_4	x_5

$x'_i = x_i - \bar{x}_i$ $i = 1, \dots, 4$ なので
平均以下だと負の値になる

第2主成分の計算式は,

$$f_2 = b_1 x'_1 + b_2 x'_2 + b_3 x'_3 + b_4 x'_4 \\ = 0.377 \times x'_1 + 0.480 \times x'_2 - 0.550 \times x'_3 - 0.387 \times x'_4 + 0.419 \times x'_5$$

x'_1 (国語), x'_2 (社会), x'_5 (英語)は, 正の重み, x'_3 (数学)と x'_4 (理科)は, 負の重み

x'_1, x'_2, x'_5 が高得点, x'_3 と x'_4 が低得点→第2主成分 f_2 の値が大きい
 x'_1, x'_2, x'_5 が低得点, x'_3 と x'_4 が高得点→第2主成分 f_2 の値が小さい

第2主成分
: 文系・理系の尺度

主成分の分散

(復習) 主成分の重みは, 分散が最大になる様に決定する.

実際には, 分散は共分散行列の固有値として求まる

第1主成分の分散 = 最大固有値

第2主成分の分散 = 2番目に大きな固有値

⋮

```
e_val, e_vec = np.linalg.eig(df_cov)
print("固有値: {}".format(e_val))
```

固有値: [141.38838586 42.82332732 7.57000962 2.76368848 4.91014428]

主成分の分散

寄与率

寄与率：元データのうち、主成分によって説明される割合
実際には、主成分の分散の合計のうち、当該主成分の分散の割合として求める

```
print("固有値:{}".format(e_val))
print(e_val)
print("固有値の合計:{}".format(sum(e_val)))
print("寄与率:{}".format(e_val/sum(e_val)))
print("固有値の累積和:{}".format(e_val.cumsum()))
print("累積寄与率:{}".format(e_val.cumsum()/e_val.sum()))
```

```
固有値: [141.38838586  42.82332732  7.57000962  2.76368848  4.91014428]
[141.38838586  42.82332732  7.57000962  2.76368848  4.91014428]
固有値の合計: 199.4555555555557
寄与率: [0.70887164  0.2147011  0.03795337  0.01385616  0.02461774]
固有値の累積和: [141.38838586 184.21171318 191.7817228 194.54541128 199.4555556]
累積寄与率: [0.70887164 0.92357274 0.9615261 0.97538226 1.]
```

寄与率

e.g. 第1主成分の寄与率=0.7089
第2主成分の寄与率=0.2147

cumsumは累積合計を求める関数
184.2117=141.3884+42.8233

累積寄与率：寄与率の累積値

e.g. 第2主成分までの累積寄与率=0.9235

第1主成分と第2主成分で原データの約92%を説明可能

利用する主成分数についての基準

絶対的な基準は無いが、累積寄与率が80%を超えるまで
主成分を採用する基準がある

```
累積寄与率: [0.70887164 0.92357274 0.9615261 0.97538226 1.]
```

第2主成分で累積寄与率が92%となり、80%を超えるので、
元の5変数の情報は、第1主成分と第2主成分にほぼ
要約されていると考えられる

scikit-learnによる主成分分析の実行（1）

sklearnのdecompositionモジュールには主成分分析を実行するPCA関数がある。

```
from sklearn.decomposition import PCA
# 主成分分析のクラスをインスタンス化
pca = PCA()
# 主成分分析
pca.fit_transform(subject)
# PCA の固有ベクトル
E_VEC=pd.DataFrame(pca.components_.T, index=subject.columns, columns=["PC{}".format(x + 1) for x in range(len(subject.columns))])
print(E_VEC)
```

	PC1	PC2	PC3	PC4	PC5
国語	-0.410118	0.377134	0.378733	-0.570911	0.469249
社会	-0.392620	0.479702	0.383711	0.626056	-0.276686
数学	-0.492834	-0.549622	-0.067016	0.393471	0.543802
理科	-0.561268	-0.387021	0.030947	-0.356427	-0.638118
英語	-0.346042	0.419269	-0.838974	-0.015969	0.018310

- fit.transform関数で主成分分析を実行した後, `components_` 属性に共分散行列の固有ベクトルが格納されている. 固有ベクトルを縦に並べたいので, `T`関数で, 固有ベクトルを転置している.
- 行見出し (index) をsubjectの列見出し (columns) とし, 列見出しを"PC1" (主成分1) から"PC5" (主成分5) としている.
- `linalg.eig`関数で求めた固有ベクトルと比べると, 任意定数 (-1) が異なる場合, 符号の違いが生じる.

scikit-learnPCA関数による主成分分析（2）

`.explained_variance_` 属性には, 固有値が格納されている.

```
# PCA の固有値
E_VAL=pd.DataFrame(pca.explained_variance_, index=["PC{}".format(x + 1) for x in range(len(subject.columns))])
print(E_VAL)
```

	0
PC1	141.388386
PC2	42.823327
PC3	7.570010
PC4	4.910144
PC5	2.763688

- `linalg.eig`関数で求めた固有値と同じ値になっていることがわかる. 固有値は主成分の分散に相当する.
- 固有値の値は大きい順 (降順) になっていることに注意
- `len`関数でsubject.columns (元のデータ) の列数を取得し, 1からその列数までの数列から1つずつ取り出し, 文字列"PC"に結合している.

scikit-learn PCA関数による主成分分析 (3)

`transform`関数を実行すると, 主成分得点が計算される.

```
pc_score=pca.transform(subject)
pd.DataFrame(pc_score, index= subject.index, columns=["PC{}".format(x + 1) for x in range(len(subject.columns))])
```

	PC1	PC2	PC3	PC4	PC5
No.					
1	-13.662350	5.089336	-0.440338	-2.322531	1.743788
2	10.111490	2.754013	2.847225	1.965365	3.520488
3	2.321167	-3.076715	-1.618398	-1.967067	0.217812
4	-17.082696	-3.538195	-3.415801	0.252072	0.566936
5	-2.446327	13.305479	0.052713	2.896265	-1.891714
6	17.588126	-0.040265	0.757300	-2.818698	-0.357254
7	5.736014	4.973499	-1.421159	-2.020334	-1.360516
8	-15.578152	-5.393213	4.540111	0.311124	-0.845588
9	4.181052	-7.180674	2.626890	0.504578	-1.670798
10	8.831676	-6.893264	-3.928544	3.199226	0.076846

主成分を計算する際の重み (因子負荷量, 共分散行列の固有ベクトル) に+-の符号の違いがあるので, 主成分得点の計算結果も, 原データと固有ベクトルの行列積を直接計算した結果 (`pr_sc`) と符号が異なる.

scikit-learn PCA関数による主成分分析 (4)

`.explained_variance_ratio_`属性には, 寄与率が格納されている.

```
# 寄与率
pd.DataFrame(pca.explained_variance_ratio_, index=["PC{}".format(x + 1) for x in range(len(subject.columns))])
```

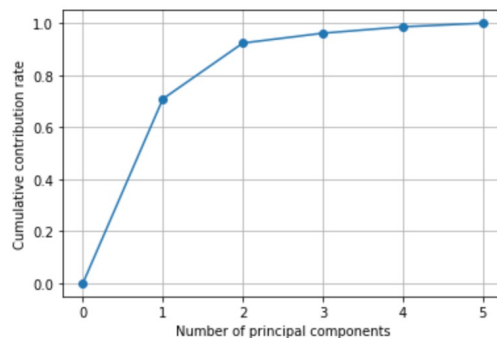
	0
PC1	0.708872
PC2	0.214701
PC3	0.037953
PC4	0.024618
PC5	0.013856

固有値が降順になっているので, 寄与率も降順になっている.
主成分 (PC) が寄与率の大きい順に並んでいる.

scikit-learn PCA関数による主成分分析 (5)

numpyのcumsum関数で寄与率の累積和を計算し、プロットする。

```
# 累積寄与率を図示する
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
plt.plot([0] + list(np.cumsum(pca.explained_variance_ratio_)), "-o")
plt.xlabel("Number of principal components")
plt.ylabel("Cumulative contribution rate")
plt.grid()
plt.show()
```

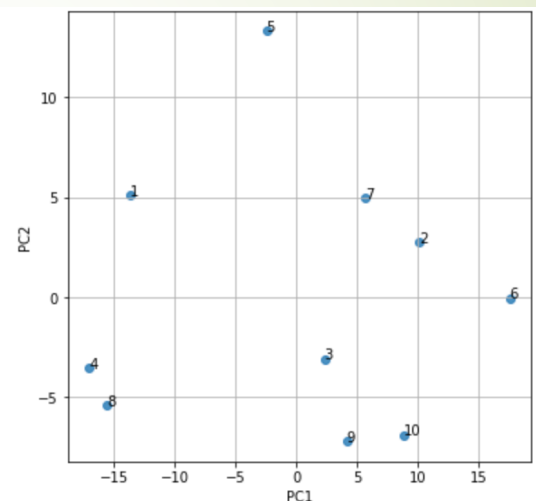


scikit-learn PCA関数による主成分分析 (6)

第1主成分と第2主成分の主成分得点の散布図

個体（生徒）の第1主成分、第2主成分の値を座標にして、生徒No.をプロットする。

```
# 主成分得点を求める
plt.figure(figsize=(6, 6))
plt.scatter(pc_score[:, 0], pc_score[:, 1], alpha=0.8)
plt.grid()
plt.xlabel("PC1")
plt.ylabel("PC2")
x = pc_score[:, 0]
y = pc_score[:, 1]
annotations = subject.index
for i, label in enumerate(annotations):
    plt.annotate(label, (x[i], y[i]))
plt.show()
```



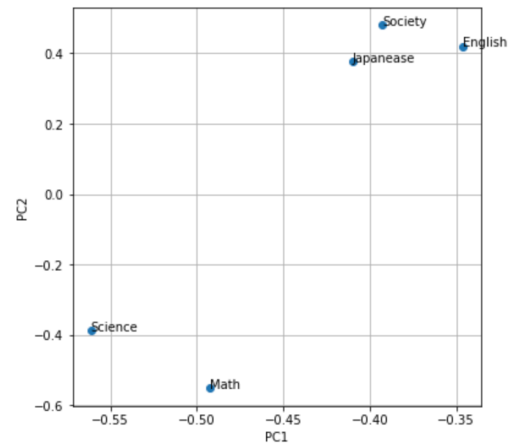
- scatter関数は、散布図を作成し、alpha引数は透明度で、0(透明)~1(不透明)の間の数値を指定する。
- enumerate関数にリスト等を渡すと、インデックス番号、要素の順に取得できる。ここでは、iがインデックス番号、labelがannotationsの要素

scikit-learn PCA関数による主成分分析 (7)

第1主成分と第2主成分を計算する際の重み（固有ベクトル）をプロットし、元の変数の相対的な関係を確認する。

```
print(E_VEC)
# 主成分負荷量を図示する
plt.figure(figsize=(6, 6))
lab_sub=["Japanese", "Society", "Math", "Science", "English"]
for x, y, name in zip(pca.components_[0], pca.components_[1], lab_sub):
    plt.text(x, y, name, fontname="MS Gothic")
plt.scatter(pca.components_[0], pca.components_[1])
plt.grid()
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()
```

	PC1	PC2	PC3	PC4	PC5
国語	-0.410118	0.377134	0.378733	-0.570911	0.469249
社会	-0.392620	0.479702	0.383711	0.626056	-0.276686
数学	-0.492834	-0.549622	-0.067016	0.393471	0.543802
理科	-0.561268	-0.387021	0.030947	-0.356427	-0.638118
英語	-0.346042	0.419269	-0.838974	-0.015969	0.018310



- `pca.components_[0]`は第1主成分, `pca.components_[1]`は第2主成分の固有ベクトルをしめす。
- `zip`関数は, `zip (A,B,C)` とした時に, A, B, Cのi番目要素を取得し, `for`ループの中で使う時に利用する。

標準化から始める主成分分析

- これまで学習した主成分分析は, 中心化（平均を引く）によるスケーリングから始めていたが, 標準化によるスケーリングから始めると, **元の変数の単位やバラツキの影響を除外できるので, 利用されることが多い。**
- 標準化によるスケーリングの場合, 共分散行列の固有値分解が相関係数行列の固有値分解となる。

```
import pandas as pd
import numpy as np
subject = pd.read_csv("subject.csv", index_col=0)
cor = np.corrcoef(subject, rowvar=False)
df_cor=pd.DataFrame(cor, index=subject.columns, columns=subject.columns)
e_val, e_vec=np.linalg.eig(df_cor)
print("固有値 : {}".format(e_val))
print("固有ベクトル")
print(e_vec)
```

```
固有値 : [3.52748736 1.03633312 0.2432075 0.13438836 0.05858367]
固有ベクトル
[[-0.48207197 -0.26970021 -0.41103483 -0.65933693 -0.30198276]
 [-0.46047733 -0.37957662 -0.40524067 0.67638778 0.1488684 ]
 [-0.39496471 0.63490442 0.07016496 0.26276767 -0.60574688]
 [-0.45070763 0.48903924 -0.00782487 -0.19569637 0.72065592]
 [-0.44320736 -0.37539363 0.81353919 -0.02074761 -0.01924512]]
```


標準化から始める主成分分析 (scikit-learn PCA関数)

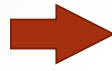
```
from sklearn import preprocessing as pp
ss = pp.StandardScaler()
subject_std=ss.fit_transform(subject)
pca = PCA()
#主成分分析
pca.fit_transform(subject_std)
pca.components_.T

array([[ -0.48207197,  0.26970021,  0.41103483, -0.65933693,  0.30198276],
       [ -0.46047733,  0.37957662,  0.40524067,  0.67638778, -0.1488684 ],
       [ -0.39496471, -0.63490442, -0.07016496,  0.26276767,  0.60574688],
       [ -0.45070763, -0.48903924,  0.00782487, -0.19569637, -0.72065592],
       [ -0.44320736,  0.37539363, -0.81353919, -0.02074761,  0.01924512]])
```

`.fit_transform`関数はデフォルトで標準化を行う。

主成分分析の計算手順

中心化による
スケーリング



原データの共分散
行列の固有値・固
有ベクトル計算



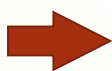
寄与率・累積寄与率の計算



主成分得点の計算

主成分の分散
因子負荷量の計算

標準化による
スケーリング



原データの相関係
数行列の固有値・
固有ベクトル計算



寄与率・累積寄与率の計算



主成分得点の計算

scikit-learn PCA関数を使う場合は、事前にデータの中心化もしくは標準化を行なっておく。
scikit-learnにあるpreprocessingクラスを使うと中心化、標準化が簡単に行える。